QML Lecture · 2일차

실습 설명 자료

segmentation 전처리 → 양자 회로 → QGAN 학습

1

[약 3분]

2일차 실습 설명을 시작하겠습니다. 오늘은 2일차 노트북 practice/day2_practice.ipynb의 내용을 코드 중심으로 설명합니다. 각 PART의 첫 장에 해당하는 노트북 섹션을 표시해 두었으므로, 노트북을 실행하면서 같은 순서로 따라오시면 됩니다. 구성은 PART 0 복습, PART 1 데이터와 전처리, PART 2 양자 회로, PART 3 QGAN 학습, PART 4 생성과 평가의 다섯 부분입니다. PART 0은 1일차 내용의 복습이고, 나머지 네 부분이 실제로 코드를 실행하는 구간입니다.

오늘은 설명과 실행을 함께 진행합니다. 저장소 첫 화면의 Open in Colab으로 노트북을 열고, 바로 파일 메뉴에서 Drive에 사본 저장을 실행해 주십시오. 사본이 없으면 실행 결과가 남지 않습니다. 'Google에서 작성하지 않았습니다'라는 경고가 뜨면 무시하고 계속을 누르면 됩니다. 런타임은 기본 CPU 그대로 둡니다. 이 회로 규모에서는 GPU로 바꿔도 빨라지지 않습니다.

셀은 위에서부터 순서대로 실행합니다. 하나를 건너뛰면 NameError가 납니다. 유휴 상태가 약 90분 이어지면 런타임이 끊겨 설치한 라이브러리와 변수가 사라지고, 이때는 다시 연결해 1번 섹션부터 실행합니다. 오류가 나면 메시지의 마지막 줄을 먼저 보십시오. 대부분 앞 셀을 실행하지 않았거나 연결이 끊긴 경우입니다.

시간 배분을 미리 말씀드리면, 가장 오래 걸리는 것은 PART 3의 학습 셀로 약 35분입니다. 학습 셀을 실행해 둔 상태에서 학습 루프, 손실 읽는 법, 계산 비용 슬라이드를 설명하겠습니다.

PART 0

1일차 복습

오늘 구현하는 QGAN의 구조와 논문 대비 축소 범위

오늘 구현하는 것	논문 ① QGAN의 patch 방식 생성자 - sub-generator 4개 + 고전 판별자
입력 데이터	HAM10000 DF 115장 → segmentation → 16×16 그레이스케일
오늘의 결과물	직접 학습한 생성자 checkpoint + 생성품질 지표 4종 표 (3일차 비교 기준)

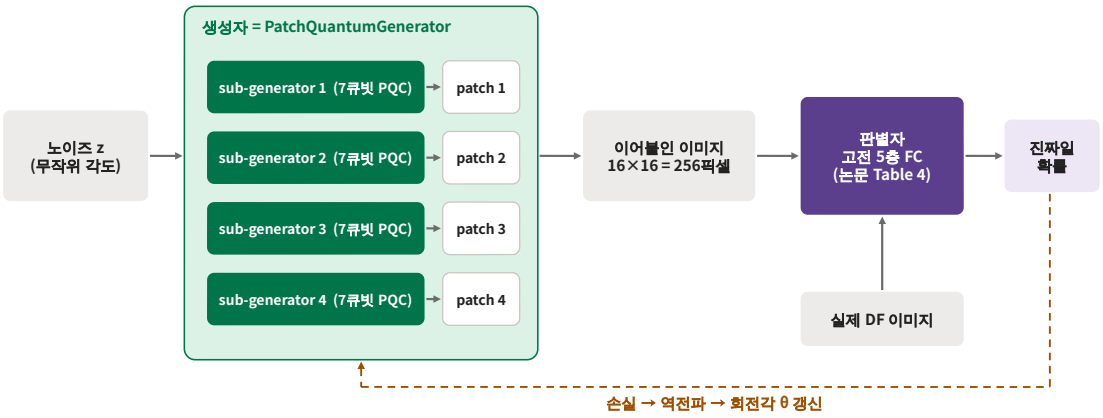
2

[약 2분]
먼저 오늘 구현하는 것을 정리합니다. 1일차에서 다룬 QGAN 논문의 구조를 코드 관점에서 다시 정리하는 것이 PART 0의 목적입니다. 오늘 구현하는 것은 논문 ①의 patch 방식 생성자, 즉 sub-generator 4개와 고전 판별자입니다. 입력 데이터는 HAM10000의 DF 115장을 segmentation 하고 16×16 그레이스케일로 줄인 것입니다. 오늘의 결과물은 직접 학습한 생성자 checkpoint와 생성 품질 지표 4종 표입니다. 이 표는 3일차에 MediQ-GAN과 비교하는 기준이 됩니다. 정리하면 오늘은 1일차의 그림을 PennyLane과 PyTorch 코드로 옮기는 날입니다.
오늘 결과물이 3일차에 어떻게 쓰이는지도 정리하겠습니다. 3일차 노트북은 저장소에 있는 강사 checkpoint, 즉 같은 구조를 600 step 학습한 생성자를 기준선으로 씁니다. 따라서 오늘 결과를 Drive에 저장하지 않아도 3일차 진행에는 문제가 없습니다. 다만 본인이 학습한 400 step 결과와 강사의 600 step 결과를 같은 지표로 비교하면 학습량이 결과에 주는 영향을 직접 볼 수 있으므로, 시간이 되면 마지막 7번 섹션에서 저장해 두시기 바랍니다.

PART 0 · 1일차 복습

QGAN 구조 : 생성자만 양자회로

patch 방식: sub-generator마다 이미지 조각(patch)을 하나씩 만들고 이어붙임



3

[약 2분]

1일차 29번에서 33번 슬라이드의 복습입니다. 구조는 비대칭입니다. 생성자는 parameterized quantum circuit, 즉 PQC 여러 개로 구성되고, 판별자는 고전 신경망으로 논문 Table 4 기준의 5층 완전연결입니다. 학습 대상은 회전 게이트의 각도 θ입니다. 판별 결과로 계산한 손실을 θ까지 역전파해 갱신하며, 그림 아래쪽의 점선 화살표가 이 흐름입니다. 오늘 코드에서는 PatchQuantumGenerator 클래스가 왼쪽의 생성자 상자 전체에 해당하고, 각 sub-generator는 7큐빗 회로 하나입니다. 큐빗 수의 근거는 PART 2에서 다시 계산합니다.

노이즈 z를 '무작위 각도'라고 적은 이유도 짚겠습니다. 고전 GAN의 노이즈는 보통 정규분포 벡터지만, 여기서는 0과 π 사이의 난수를 RY 게이트의 회전각으로 넣습니다. 노이즈가 회로의 입력 상태를 정하고, 학습된 θ가 그 상태를 변환하는 구조입니다.

확인 질문을 하나 드리겠습니다. 이 구조에서 학습되는 것은 무엇이고, 학습되지 않는 것은 무엇입니까. 답은 이렇습니다. 학습되는 것은 생성자의 회전각 θ와 판별자의 가중치입니다. 노이즈를 넣는 RY의 각도는 매번 새로 뽑는 입력이라 학습 대상이 아닙니다.

PART 0 · 1일차 복습

논문과 오늘 실습의 차이

회로의 형태(RY encoding + RY·CZ variational layer + post-selection)는 같고, 규모와 학습량을 줄임

항목	논문 (Andra et al.)	2일차 실습	이유
sub-generator	16개	4개	2·4·8·16개 실측 중 품질 최고
큐빗 (sub-generator당)	데이터 7 + ancilla 1	데이터 6 + ancilla 1	$256 = 4 \times 2^6$
회로 depth	10	10	논문과 동일
판별자	5층 FC (Table 4)	5층 FC (Table 4)	논문과 동일
학습량	2,000 epoch (1 ~ 1.5시간)	400 step (Colab 약 35분)	수업 시간 안에 완료
실행 환경	GPU	Colab CPU, exact 시뮬레이션	설치·계정 불필요

논문 설정인 sub-generator 16개로 학습하면 판별자가 이겨 loss_d가 0.003까지 떨어짐. 용량을 늘려도 품질이 오르지 않았음.

논문 값: Andra et al. 프리프린트 Table 3 · 실습 값: README sub-generator 실험

[약 2분]

논문과 오늘 실습의 차이를 표로 정리했습니다. 저장소 README의 '원 논문과 다른 점' 표와 같은 내용입니다. sub-generator는 논문이 16개, 실습이 4개입니다. 처음에는 논문과 큐빗 수가 가까운 2개로 정했는데, 2, 4, 8, 16개를 같은 조건, 즉 600 step과 depth 10에서 실측해 품질이 가장 좋은 4개로 바꿨습니다. 논문 설정인 16개에서는 판별자가 이겨 loss_d가 0.003까지 떨어졌고, 용량을 늘려도 품질이 오르지 않았습니다. sub-generator당 큐빗은 데이터 6개와 ancilla qubit 1개로, 256픽셀을 4개 patch로 나누기 때문입니다. 회로 depth 10과 5층 판별자는 논문과 같습니다.

학습량은 논문이 2,000 epoch이고 실습은 400 step으로 Colab에서 약 35분입니다. 실습은 exact 상태벡터 시뮬레이션이라 shot noise와 하드웨어 오류가 없다는 점도 기억해 주십시오.

이 차이가 결과 해석에 주는 의미를 덧붙이겠습니다. 오늘 결과가 논문보다 나쁘게 나와도 그것이 논문 주장을 반박하는 근거가 되지는 않습니다. 해상도, 학습량, sub-generator 수가 모두 다르기 때문입니다. 반대로 실습에서 어떤 경향이 보여도 그대로 논문 조건에 일반화할 수는 없습니다. 오늘 실습의 목적은 같은 구조가 어떻게 동작하는지 코드 수준에서 확인하는 것이고, 성능 비교는 3일차에 같은 조건의 두 모델 사이에서만 합니다.

PART 1 · 노트북 §2 ~ §3

데이터와 segmentation 전처리

HAM10000에서 DF·NV를 고르고, 병변만 남겨 16×16으로 줄이는 과정

노트북 범위	§2 EDA, §3 segmentation 전처리
다루는 코드	segment_fcm_kmeans · resize_for_encoding · preprocess_class
논문 근거	Andra et al. Section 3.1 (K-means + Fuzzy C-means), Table 10

5

[약 2분]

PART 1은 노트북 §2와 §3입니다. EDA와 segmentation 전처리를 다룹니다. 코드는 segment_fcm_kmeans, resize_for_encoding, preprocess_class 세 함수입니다. 논문 근거는 Andra 등의 Section 3.1, K-means와 Fuzzy C-means를 결합한 방식과 Table 10입니다.

§2를 실행할 때 알아 둘 점입니다. HAM10000 원본은 kagglehub로 Colab 런타임에 직접 내려받으며 약 6GB입니다. 개인 PC나 Drive에는 저장되지 않습니다. 경로가 /kaggle/input으로 시작해도 정상입니다. Kaggle의 Colab 캐시를 바로 연결한 경우입니다.

공개 데이터셋이라 보통 로그인 없이 받아집니다. 다운로드 셀에서 401이나 403 인증 오류가 날 때만 Kaggle 설정 화면에서 API 토큰을 발급해 로그인 셀을 실행합니다. 발급 화면 캡처는 노트북에 있습니다. 토큰은 비밀번호처럼 다루고 노트북에 적거나 공유하지 않습니다.

PART 1 · 노트북 § 2

범위 축소 : DF vs NV

7개 클래스 중 가장 적은 DF를 증강 대상으로, 가장 많은 NV를 비교 대상으로 둠

클래스	이미지 수	비율	실습에서의 역할
DF (피부섬유종)	**115**	1.1%	증강 대상 - 생성자 학습 데이터
NV (모반)	6,705	66.9%	비교 대상 - 지표 ② 기준 (345장 사용)
전체 7개 클래스	10,015	100%	DF : NV = 1 : 58

```
metadata = pd.read_csv(f"{DATA_DIR}/HAM10000_metadata.csv")
counts = metadata["dx"].value_counts()

# df     115   <- 증강 대상
# nv    6705   <- 비교 대상
```

- metadata는 image_id와 진단명(dx)을 연결하는 표임
- 이미지 파일은 part_1·part_2 두 폴더에 나뉘어 있어 find_image_path가 양쪽을 확인함

HAM10000 메타데이터 (Tschandl et al., Scientific Data 5, 180161, 2018)

[약 2분]

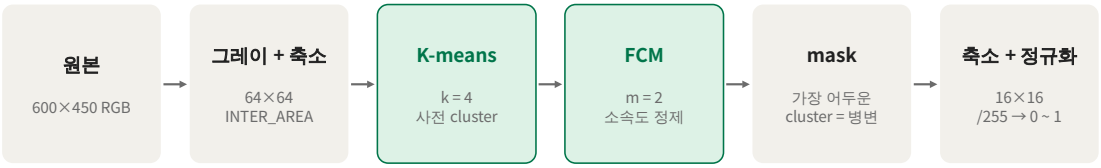
DF와 NV의 범위를 정리합니다. DF가 115장, NV가 6,705장으로 약 1 대 58의 불균형입니다. 증강 대상은 DF입니다. NV는 3일차까지 클래스 평균 상관 지표에서 비교용으로만 쓰며 345장을 사용합니다. 코드를 보면 metadata는 image_id와 진단명 dx를 연결하는 표이고, value_counts()는 진단명별 이미지 수를 많은 순으로 반환합니다. 이미지 파일은 part_1과 part_2 두 폴더에 나뉘어 있어서 find_image_path 함수가 양쪽을 모두 확인합니다. 실행하면 df 115, nv 6,705가 출력되는지 먼저 확인해 주십시오. 이 숫자가 맞아야 이후 전처리가 올바른 클래스를 대상으로 동작합니다.

두 클래스만 쓰는 이유는 논문 설정을 따르기 위해서입니다. Andra 등은 가장 적은 DF와 가장 많은 NV만 골라 이진 분류 문제로 만들었습니다. 1 대 58의 불균형은 생성모델로 소수 클래스를 보충하려는 동기 그 자체입니다. NV 345장은 DF 115장의 세 배로, 평균 이미지를 안정적으로 만들 만큼만 씁니다.

PART 1 · 노트북 § 3

전처리 pipeline

원본 → 병변만 남긴 64×64 → 16×16, 0 ~ 1. 저장소 자산(df_preprocessed.npy)도 같은 조건으로 만들어짐



논문 Section 3.1의 K-means + Fuzzy C-means hybrid segmentation

병변만 남기는 이유

논문 Table 10: raw 이미지로 증강하면 분류 성능이 오히려 떨어지고, segmentation을 거친 이미지에서만 개선됨

64×64를 거치는 이유

FCM 비용이 픽셀 수에 비례함. 원본 27만 픽셀 대신 4,096픽셀로 clustering해 1장당 수 초로 줄임

7

[약 3분]

전처리 pipeline입니다. 원본 600×450 RGB를 그레이로 바꾸고 64×64로 줄인 뒤, K-means와 Fuzzy C-means로 마스크를 만들고, 마지막으로 16×16으로 줄이면서 255로 나눠 0과 1 사이로 맞춥니다. 병변만 남기는 이유는 논문 Table 10에 있습니다. raw 이미지로 증강하면 분류 성능이 오히려 떨어지고, segmentation을 거친 이미지에서만 개선되었습니다. 64×64를 먼저 거치는 이유는 비용입니다. FCM의 비용은 픽셀 수에 비례하고, 600×450 원본은 약 27만 픽셀이라 한 장에 수십 초가 걸리지만 4,096픽셀이면 수 초로 줄어듭니다. 가장 어두운 cluster를 병변으로 보는 것은 피부경 이미지에서 병변이 주변 피부보다 어둡다는 가정이고, 털이나 그림자가 많은 이미지에서는 틀릴 수 있는 한계가 있습니다. 마지막 255로 나누는 단계가 전처리 데이터의 0과 1 척도, 즉 unit 척도의 기준이 됩니다.

노트북에 적힌 한계도 하나 더 있습니다. 가장 어두운 cluster를 병변으로 보기 때문에, 피부경 이미지 모서리의 검은 테두리가 병변 대신 잡힐 수 있습니다. 3일차 노트북에서 실제 DF 이미지를 보면 segmentation이 테두리만 남긴 경우가 실제로 있습니다.

확인 질문입니다. 16×16으로 줄이기 전에 배경을 지우는 이유는 무엇입니까. 답은 줄인 뒤에는 병변 픽셀과 배경 픽셀이 한 칸에 평균되어 섞이므로, 그 전에 배경을 0으로 만들어 두어야 병변 정보가 남기 때문입니다.

PART 1 · 노트북 §3 코드

segment_fcm_kmeans 코드

밝기 1채널만 써서 4개 cluster로 나누고, 가장 어두운 cluster를 남김

```
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
gray = cv2.resize(gray, (64, 64), interpolation=cv2.INTER_AREA)
pixels = gray.reshape(-1, 1).astype(np.float32)      # 1

km = KMeans(n_clusters=4, n_init=4, random_state=0)  # 2
km = km.fit(pixels)
dist = np.abs(pixels - km.cluster_centers_.reshape(1, -1))
u_init = 1.0 / (dist + 1e-6) ** 2                    # 3
u_init = (u_init / u_init.sum(axis=1, keepdims=True)).T

cntr, u, *_ = fuzz.cluster.cmeans(pixels.T, c=4, m=2.0,
                error=1e-4, maxiter=100, init=u_init)
membership = np.argmax(u, axis=0)                    # 4
lesion = np.argmin(cntr.flatten())
mask = ((membership == lesion).reshape(64, 64) * 255).astype(np.uint8)
return cv2.bitwise_and(gray, gray, mask=mask)        # 5
```

① 밝기만 사용
위치 정보 없이 (픽셀 수, 1)로 clustering

② K-means 사전 cluster
k-means++ 초기화 4회 중 최선 채택

③ FCM 초기 소속도
중심 거리의 역제곱 - 수렴이 빠르고 재현 가능

④ 병변 cluster 선택
중심이 가장 어두운 cluster = 병변

⑤ mask 적용
병변은 원래 밝기 유지, 배경은 0

8

[약 2분]
segment_fcm_kmeans 함수를 다섯 부분으로 나누어 읽겠습니다. ①은 밝기만 사용하는 부분입니다. 위치 정보 없이 픽셀 밝기만으로 clustering하므로 pixels의 모양은 픽셀 수 곱하기 1입니다. ②는 K-means 사전 cluster입니다. sklearn KMeans의 기본 초기화는 k-means++이고, n_init이 4이면 초기화를 네 번 바꿔 inertia, 즉 cluster 내 거리 제곱합이 가장 낮은 결과를 씁니다. ③은 FCM 초기 소속도입니다. K-means 중심까지 거리의 역제곱으로 소속도를 만드는데, m이 2일 때 FCM의 소속도 공식과 같은 형태이며 무작위 시작보다 빨리 수렴하고 결과가 매번 같습니다. ④는 FCM 결과의 소속도가 가장 큰 cluster로 hard 라벨을 정한 뒤, 중심 밝기가 가장 낮은 cluster를 병변으로 보는 부분입니다. ⑤의 bitwise_and는 x와 x의 AND가 x이므로 연산 효과는 없고 mask 효과만 남깁니다.
노트북에서는 DF 첫 이미지를 64×64로 줄여 이 함수의 결과를 바로 그려 봅니다. random_state를 0으로 고정했고 FCM의 초기 소속도도 K-means 결과에서 만들기 때문에, 같은 이미지에서는 항상 같은 마스크가 나옵니다. 결과 그림에서 병변이 아닌 영역, 예를 들어 모서리가 남아 있다면 앞에서 말씀드린 테두리 한계의 사례입니다.

PART 1 · 노트북 § 3

K-means와 Fuzzy C-means

K-means는 픽셀마다 cluster 하나, FCM은 cluster별 소속도를 나눠 가짐

	K-means	Fuzzy C-means (FCM)
배정 방식	hard - 픽셀마다 cluster 1개	soft - 소속도 u (픽셀별 합 = 1)
중심 갱신	배정된 픽셀의 평균	소속도^m 가중 평균
주요 파라미터	n_clusters, n_init	c, m (fuzziness), error, maxiter, init
이 코드에서의 역할	FCM 초기값을 만드는 사전 cluster	소속도 정제 → argmax로 최종 라벨
출력	cluster_centers_, labels_	cntr, u, u0, d, jm, p, fpc

m (fuzziness)

- m → 1: K-means처럼 딱 잘라 나뉨
- m이 클수록 소속이 여러 cluster로 퍼짐
- 보통 2를 씀 (이 코드도 2)

fpc (0 ~ 1)

분할 선명도. 1에 가까울수록 명확하고, 1/c(여기서는 0.25)에 가까울수록 모호함

9

[약 2분]

K-means와 Fuzzy C-means를 비교합니다. K-means는 픽셀마다 cluster 하나를 배정하는 hard 방식이고, FCM은 픽셀별로 합이 1인 소속도를 갖는 soft 방식입니다. 경계 픽셀, 즉 병변 가장자리는 어느 한쪽으로 딱 나누기 어렵습니다. FCM은 병변 60%, 피부 40%처럼 표현해 경계가 덜 거칠어집니다. 다만 이 코드는 마지막에 argmax로 hard 라벨을 다시 만들기 때문에, FCM의 이점은 경계 픽셀의 배정이 더 안정적이라는 데 있습니다. 오른쪽 상자의 m은 fuzziness 지수입니다. m이 1에 가까우면 K-means처럼 딱 잘라 나뉘고, 클수록 소속이 여러 cluster로 퍼지며 보통 2를 쓰고 이 코드도 2입니다. fpc는 분할 선명도로, 1에 가까울수록 명확하고 cluster 수의 역수인 0.25에 가까울수록 모호하므로 이미지별 품질 점검에 쓸 수 있습니다.

확인 질문입니다. 마지막에 argmax로 hard 라벨을 다시 만든다면 FCM을 쓰는 의미가 무엇입니까.

답은 cluster 중심의 추정에 있습니다. FCM은 소속도의 m제곱을 가중치로 중심을 다시 계산하므로, 경계 픽셀이 여러 cluster의 중심에 부분적으로 반영됩니다. 그 결과 중심 밝기가 달라지고, 최종 argmax 배정도 K-means만 쓴 경우와 달라질 수 있습니다.

PART 1 · 노트북 § 3

16×16 해상도와 축소 방법

픽셀 수가 큐빗 수를 정함. 256픽셀 = sub-generator 4개 × 2^6



16 × 16 = 256픽셀

큐빗 수 계산

256 ÷ 4 = 64픽셀/patch → $\log_2 64 = 6$ 데이터 큐빗 → + ancilla 1 = 7큐빗

보간법	방식	16×16 축소에서
INTER_AREA	출력 픽셀이 덮는 영역의 평균	사용 - 저장소 자산과 같은 조건
INTER_LINEAR	주변 2×2 선형 보간 (기본값)	큰 축소에서 앨리어싱
INTER_NEAREST	가장 가까운 픽셀 1개 복사	마스크용, 밝기 정보 손실

10

[약 2분]

16×16 해상도와 축소 방법입니다. 이 해상도는 patch 구조에서 정해집니다. 256픽셀을 sub-generator 4개로 나누면 patch당 64픽셀이고, 측정 결과의 basis state 2의 n제곱 개가 픽셀 n개에 대응하므로 64의 log2인 6이 데이터 큐빗 수가 됩니다. 여기에 ancilla qubit 1개를 더해 7큐빗입니다. 생성자 출력 256개는 patch 순서대로 이어 붙이므로 16×16으로 reshape하면 patch 하나가 가로 16, 세로 4의 띠가 됩니다. 아래 표는 축소 보간법 비교입니다. INTER_AREA는 출력 픽셀이 덮는 원본 영역의 평균이라 크게 줄일 때 정보가 고르게 반영됩니다. INTER_LINEAR는 주변 2×2만 참조해서 축소 폭이 크면 작은 병변이 사라지거나 앨리어싱이 생깁니다. INTER_NEAREST는 마스크용이며 밝기 정보가 손실됩니다. 이 저장소의 자산은 INTER_AREA 조건으로 만들었습니다.

노트북에는 IMG_SIZE를 8로 바꾸는 선택지도 적혀 있습니다. 8×8이면 회로가 작아져 더 빨리 돌지만, 3일차 자산이 16×16이라 오늘 결과를 3일차와 비교할 수 없게 됩니다. 수업에서는 16을 유지해 주십시오.

축소 보간법은 CLAUDE.md의 규칙이기도 합니다. 노트북에서 직접 전처리한 결과와 저장소 자산이 같은 조건이어야 둘을 섞어 써도 결과가 일치하므로, 양쪽 모두 INTER_AREA로 맞춰 두었습니다.

PART 2 · 노트북 §4 ~ §5

양자 회로 : sub-generator 하나

노이즈 encoding → variational layer 반복 → 측정 → post-selection

노트북 범위	§4 회로 1개 출력 확인, §5 생성자·판별자 정의
다루는 코드	sub_generator_circuit · probs_all[::2] · PatchQuantumGenerator

11

[약 1분]

PART 2는 노트북 §4와 §5입니다. §4는 학습 없이 회로 출력만 확인하는 섹션이고, §5의 앞부분에서 생성자와 판별자를 정의합니다. 다루는 코드는 sub_generator_circuit 함수, probs_all[::2] 슬라이싱, PatchQuantumGenerator 클래스입니다.

§4를 실행할 때 확인할 것은 두 가지입니다. 첫째, 학습 전 회로의 출력은 무작위 패턴이 정상입니다. 회전각이 아직 난수이기 때문입니다. 둘째, post-selection으로 고른 64개 확률의 합이 1이 아니라 0과 1 사이의 값으로 나오는지 보십시오. 다음 두 장에서 이 값이 왜 1이 아닌지, 그리고 그것이 왜 필요한지를 설명합니다.

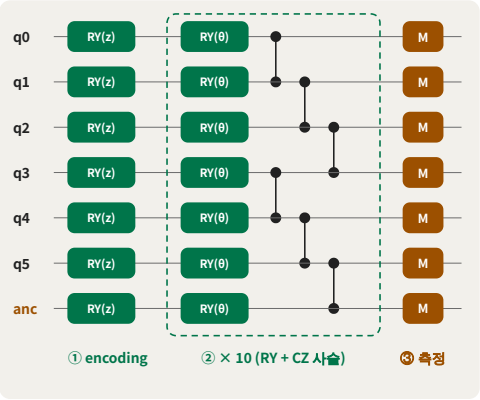
PART 2 · 노트북 §4 코드

sub-generator 회로

RY로 노이즈를 넣고, [학습 RY + 인접 CZ]를 10번 반복한 뒤 7큐빗 전체를 측정

```
@qml.qnode(dev)
def sub_generator_circuit(noise, weights):
    for i in range(N_QUBITS):           # 1 encoding
        qml.RY(noise[i], wires=i)
    for layer in range(Q_DEPTH):        # 2 x 10
        for i in range(N_QUBITS):
            qml.RY(weights[layer, i], wires=i)
        for i in range(N_QUBITS - 1):
            qml.CZ(wires=[i, i + 1])
    return qml.probs(wires=range(N_QUBITS))  # 3
```

weights 모양 (10, 7) - sub-generator 하나당 학습 파라미터 70개



12

[약 2분]

sub-generator 회로 하나를 코드와 회로도로 읽겠습니다. 첫째, encoding입니다. 샘플마다 0과 π 사이의 난수 각도를 RY 게이트로 넣으며, 같은 가중치에서 다른 이미지가 나오는 원천이 이 노이즈입니다. 둘째, 학습 부분입니다. RY 회전과 인접 큐빗 사이의 CZ를 depth 10만큼 반복합니다. RY는 상태의 진폭을 실수로 유지한 채 회전시켜 측정 확률을 직접 바꾸는데, 확률이 곧 픽셀 밝기이므로 RY만으로 충분합니다. CZ는 인접 큐빗을 얽어 픽셀 사이의 상관을 만듭니다. CZ가 없으면 각 큐빗이 독립이라 확률분포가 곱 형태로 제한됩니다. 셋째, 측정입니다. qml.probs는 2의 7제곱, 즉 128개 basis state의 확률을 반환합니다. weights의 모양이 10 곱하기 7이므로 sub-generator 하나당 학습 파라미터는 70개입니다.

확인 질문입니다. depth를 10에서 20으로 늘리면 무엇이 바뀝니까. 학습 파라미터는 층마다 7개씩이므로 sub-generator당 70개에서 140개로 두 배가 되고, 게이트 수도 두 배라 회로 한 번의 실행 시간도 늘어납니다. 1일차에서 다룬 것처럼 회로가 깊어질수록 gradient가 작아지는 barren plateau의 위험도 커지므로, depth는 표현력과 학습 가능성 사이의 절충입니다.

PART 2 · 노트북 § 4 코드

ancilla qubit과 post-selection

128개 확률 중 ancilla qubit = 0인 64개만 patch로 씀. 코드로는 probs_all[::2]

인덱스	q0 … q5 (데이터)	anc	사용
0	000000	0	**patch 픽셀 0**
1	000000	1	버림
2	000001	0	**patch 픽셀 1**
3	000001	1	버림
…	…	…	…
126	111111	0	**patch 픽셀 63**
127	111111	1	버림

```
probs_all = sub_generator_circuit(noise, weights)
output = probs_all[::2]   # even index = anc 0
```

ancilla 없이 측정하면

- 64개 확률의 합 = 항상 1
- 배경 patch(합 ≈ 0)와 병변 patch(합이 큼)를 구분해 표현할 수 없음

ancilla = 0만 선택하면

- 64개 확률의 합 = 0 ~ 1 사이에서 자유로움
- patch별 밝기 총합도 학습 대상이 됨
- 회로는 그대로, 측정·해석만 바꾼 것

13

[약 3분]
ancilla qubit과 post-selection입니다. 128개 확률 중 ancilla가 0인 64개만 patch 픽셀로 씁니다. 표를 보면 인덱스 0은 데이터 000000에 ancilla 0이어서 patch 픽셀 0이 되고, 인덱스 1은 ancilla가 1이라 버립니다. 짝수 인덱스만 쓰는 이유는 PennyLane이 wire 0을 최상위 비트, 마지막 wire를 최하위 비트로 두기 때문입니다. 마지막 wire가 ancilla이므로 인덱스가 짝수일 때 ancilla가 0이고, 코드의 probs_all[::2]가 이 선택입니다. 왼쪽 상자의 이유를 보겠습니다. ancilla 없이 데이터 큐빗만 측정하면 64개 확률의 합이 항상 1입니다. 그러면 배경이 대부분이라 합이 0에 가까워야 하는 patch를 표현할 수 없습니다. ancilla가 0인 항목만 쓰면 합이 0과 1 사이에서 자유로워지고 이 밝기 총합도 학습 대상이 됩니다. 3일차 노트북 §6 ①에서 이 측정 방식으로 바꾸자 생성 이미지 평균과 실제 평균의 상관계수가 0.00에서 0.49로 올랐습니다. 8×8 단계의 사전 실험 결과입니다. 일반적인 ancilla의 용도인 오류 정정 신드롬 측정 등과는 목적이 다르다는 점에 유의하십시오.
확인 질문입니다. ancilla qubit을 두 개로 늘리면 코드는 어떻게 바뀝니까. 하위 두 비트가 ancilla가 되므로 두 ancilla가 모두 0인 항목은 네 칸마다 하나입니다. 3일차 노트북의 생성자 코드에서는 이 간격을 anc_stride, 즉 2의 ancilla 수 제곱으로 계산해 일반화해 두었습니다.

PART 2 · 노트북 §5 코드

PatchQuantumGenerator : patch 4개를 이어 한 장으로

sub-generator별로 batch 크기만큼 회로를 실행하고, patch를 가로로 이어붙임

```
class PatchQuantumGenerator(nn.Module):
    def __init__(self, n_generators, n_data_qubits, ...):
        q_dev = qml.device("default.qubit", ...)
        @qml.qnode(q_dev, interface="torch",
                   diff_method="backprop")
        def circuit(noise, weights): ...   # same as §4
        self.q_params = nn.ParameterList([
            nn.Parameter(torch.rand(q_depth, n_qubits) * pi)
            for _ in range(n_generators)])

    def forward(self, batch_size):
        patches = []
        for g in range(self.n_generators):      # 4
            rows = [self.circuit(torch.rand(7) * pi,
                      self.q_params[g])[::2]
                    for _ in range(batch_size)] # 8
            patches.append(torch.stack(rows))
        return torch.cat(patches, dim=1)   # (batch, 256)
```

gen(8) 한 번의 실행

sub-gen 1	× 8 샘플	(8, 64)
sub-gen 2	× 8 샘플	(8, 64)
sub-gen 3	× 8 샘플	(8, 64)
sub-gen 4	× 8 샘플	(8, 64)

torch.cat(dim=1) → (8, 256)

회로 실행 4 × 8 = 32번

14

[약 2분]

PatchQuantumGenerator 클래스입니다. 회로는 §4와 같지만 device가 default.qubit이고 diff_method가 backprop입니다. PyTorch 자동미분으로 회전각까지 gradient를 계산하기 위한 설정이며, lightning.qubit은 backprop을 지원하지 않습니다. forward 함수의 이중 루프가 비용의 핵심입니다. sub-generator 4개마다 batch 8개 샘플을 각각 실행하므로 한 번의 gen() 호출에 회로가 32번 실행됩니다. 각 sub-generator의 결과를 torch.cat으로 dim 1 방향으로 이어 붙이면 batch 곱하기 256의 텐서가 되고, 이미지로 볼 때는 reshape로 16×16을 만듭니다. 오른쪽 그림이 이 실행 구조입니다. q_params는 sub-generator마다 depth 곱하기 큐빗 모양의 파라미터를 0과 π 사이의 난수로 초기화한 것입니다.

3일차 노트북에도 같은 클래스가 다시 나옵니다. 그때는 생성자의 구조를 checkpoint에 저장된 config, 즉 sub-generator 수, 큐빗 수, depth로 다시 만든 뒤 가중치를 불러옵니다. 구조가 같아야 저장된 q_params의 모양이 맞기 때문입니다.

코드에서 한 가지 더 보면, 노이즈는 샘플마다 torch.rand로 새로 뽑아 π를 곱합니다. 같은 sub-generator라도 batch 안의 샘플마다 서로 다른 입력 상태에서 출발하므로 서로 다른 patch가 나옵니다.

PART 2 · 노트북 §5 코드

파라미터 수 : 생성자 280개, 판별자 19,201개

약 70배 차이가 나서 학습률을 다르게 둠 (생성자 0.05, 판별자 0.001)

구성	모양	파라미터
생성자: 회전각 RY(θ)	**sub-generator 4 × depth 10 × 큐빗 7**	**280**
판별자 Linear 1	256 → 64	16,448
판별자 Linear 2	64 → 32	2,080
판별자 Linear 3	32 → 16	528
판별자 Linear 4	16 → 8	136
판별자 Linear 5 + Sigmoid	8 → 1	9
판별자 합계		**19,201**

```
opt_g: lr=0.05
opt_d: lr=0.001
```

판별자가 약 70배 크고 빨리 배우므로, 생성자 학습률을 크게, 판별자는 작게 둬 균형을 맞춤.

15

[약 2분]

파라미터 수를 확인합니다. 생성자의 학습 파라미터는 sub-generator 4개, depth 10, 큐빗 7개를 곱한 280개입니다. noise encoding의 RY는 입력값이므로 학습 대상이 아닙니다. 판별자는 256에서 64, 32, 16, 8, 1로 줄어드는 완전연결이고, 첫 층 하나가 16,448개로 대부분을 차지해 합계가 19,201개입니다. 판별자가 생성자보다 약 70배 크고 빨리 배우므로, 생성자의 학습률을 0.05로 크게, 판별자는 0.001로 작게 두어 균형을 맞춥니다. 참고로 3일차의 MediQ-GAN은 Adam의 betas가 0.5와 0.999, 학습률 2×10^{-4}라는 GAN 관례값을 씁니다. 두 모델의 학습 설정이 다르다는 점도 비교 때 고려할 대상입니다.

판별자 첫 층의 16,448개는 입력 256에 출력 64를 곱한 가중치 16,384개와 bias 64개를 더한 값입니다. 학습률은 모델마다 따로 맞춰야 한다는 점도 강조하겠습니다. README의 학습 안정성 항목을 보면, QGAN에서 쓰던 생성자 학습률 0.02를 MediQ-GAN에 그대로 쓰자 3 epoch 만에 판별자 손실이 0.0027까지 떨어지고 mode collapse가 일어났습니다. 같은 GAN이라도 생성자의 구조가 다르면 적절한 학습률이 다릅니다.

PART 3 · 노트북 §5

QGAN 학습

학습 루프, 손실 읽는 법, 계산 비용과 한계

노트북 범위	§5 학습 셀 (400 step)
다루는 코드	학습 루프 (판별자 step · 생성자 step) · BCE 손실
소요 시간	Colab CPU 약 35분, 진행 로그는 20 step마다

16

[약 2분]

PART 3은 QGAN 학습입니다. 노트북 §5의 학습 셀로 400 step을 실행합니다. 다루는 코드는 학습 루프, 즉 판별자 step과 생성자 step, 그리고 BCE 손실입니다. Colab CPU에서 약 35분이 걸리고 진행 로그는 20 step마다 출력됩니다. 로그의 숫자를 읽는 기준은 이 PART의 손실 기준값 슬라이드에서 설명합니다.

학습 셀을 실행하기 전에 확인할 점입니다. 학습 셀은 진행 로그를 20 step마다 출력하고, 경과 시간과 남은 예상 시간도 함께 보여 줍니다. 시간이 부족하면 STEPS를 300으로 줄여도 됩니다. 학습 중에는 다른 셀을 실행하지 마십시오.

실행 중인 셀은 왼쪽의 ■ 버튼으로 중단할 수 있습니다. 중단하면 그 시점까지 학습된 생성자가 그대로 남으므로, 다음 섹션의 생성과 평가를 이어서 실행할 수 있습니다. 이 경우 결과는 학습량이 적은 만큼 달라집니다. 이제 학습 셀을 실행해 두고 이어지는 슬라이드를 설명하겠습니다.

PART 3 · 노트북 §5 코드

학습 루프 : 한 step에 두 번 갱신

판별자를 먼저 갱신하고, 같은 step에서 생성자를 갱신함

```
for step in range(STEPS):                    # 400
    batch = real_flat[torch.randint(0, 115, (8,))]

    # discriminator: real -> 1, fake -> 0
    opt_d.zero_grad()
    loss_d = bce(disc(batch), ones) \
           + bce(disc(gen(8).detach()), zeros)
    loss_d.backward(); opt_d.step()

    # generator: make disc say 1 for fakes
    opt_g.zero_grad()
    loss_g = bce(disc(gen(8)), ones)
    loss_g.backward(); opt_g.step()
```

① 판별자 step

실제 → 1, 생성 → 0. detach로 생성자 쪽 기울기는 계산하지 않음

② 생성자 step

생성 이미지에 라벨 1을 붙여 손실 계산. 판별자를 속이는 방향으로 θ 갱신

③ gen() 호출 2번

판별자용 1번 + 생성자용 1번. step당 회로 64번 실행

17

[약 2분]

학습 루프를 세 부분으로 읽겠습니다. 판별자 step은 실제 batch는 1, 생성 이미지는 0으로 맞히도록 학습하는 단계입니다. 생성 이미지에는 detach를 적용해 생성자 쪽 계산 그래프를 끊으므로 판별자만 갱신됩니다. 생성자 step은 생성 이미지에 정답 라벨 1을 붙여 손실을 계산하는 단계입니다. 1일차에서 말씀드린 non-saturating 손실이며, 판별자를 속이는 방향으로 회전각이 갱신됩니다. 마지막으로 gen()을 두 번 호출하는 점입니다. 판별자용 한 번과 생성자용 한 번이어서, step당 회로가 64번 실행됩니다. 실제 이미지는 복원추출로 8장씩 뽑고, 115장이므로 1 epoch은 약 14 step에 해당합니다.

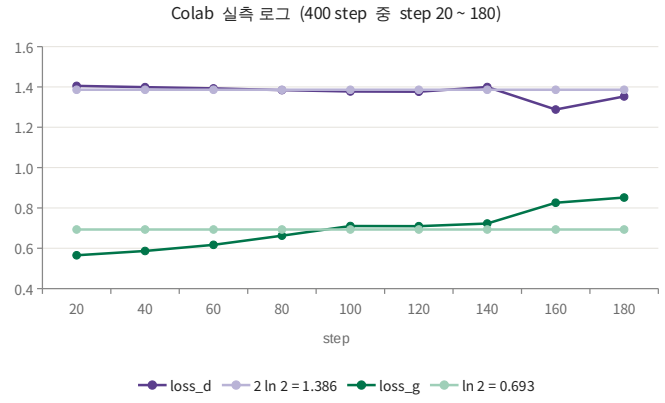
학습량을 epoch으로 환산하면 이렇습니다. 115장을 8장씩 뽑으므로 1 epoch은 약 14 step이고, 400 step은 약 28 epoch에 해당합니다. 논문의 2,000 epoch과 비교하면 매우 짧은 학습입니다.

확인 질문입니다. 판별자 step에서 detach를 빼면 어떻게 됩니까. 판별자 손실의 backward가 생성 이미지를 거쳐 양자회로 32개까지 거슬러 올라가 회전각의 gradient를 계산합니다. opt_d는 판별자 파라미터만 갱신하고 생성자 step 앞의 opt_g.zero_grad()가 이 gradient를 지우므로 결과는 같지만, 쓰이지 않을 회로 미분을 매 step 한 번 더 하게 됩니다. detach는 이 낭비를 막는 역할입니다.

PART 3 · 진행 로그 읽기

손실 기준값 : 1.386과 0.693

판별자가 진짜·가짜를 구분하지 못할 때(D = 0.5)의 BCE 값이 기준선



값	의미
loss_d ≈ 1.386	구분 불가 - 경쟁 유지
loss_d → 0	판별자 완승, 생성자 신호 소실
loss_g ≈ 0.693	생성 이미지를 반반으로 판정
loss_g = 0.85	D(G(z)) ≈ e^−0.85 ≈ 0.43

GAN 손실은 단조 감소하지 않음. 손실이 균형처럼 보여도 품질은 낮을 수 있어 §6 지표로 따로 평가함.

18

[약 2분]

진행 로그를 읽는 기준값입니다. loss_d는 −[log D(x) + log(1 − D(G(z)))]이므로, 판별자가 모든 출력을 0.5로 내면 2 ln 2, 약 1.386입니다. 이 값은 실제와 생성을 구분하지 못해 경쟁이 유지되는 상태입니다. loss_d가 0으로 가면 판별자가 완승한 것이고 생성자가 받는 학습 신호가 소실됩니다. loss_g는 −log D(G(z))이므로 D(G(z))가 0.5이면 ln 2, 약 0.693이고, 거꾸로 D(G(z))는 loss_g의 마이너스를 지수로 취한 값입니다. 예를 들어 loss_g가 0.85이면 D(G(z))는 약 0.43입니다. 오른쪽 차트는 Colab 실행 로그의 step 20부터 180까지입니다. loss_g가 0.57에서 0.85로 오른 것은 판별자가 생성 이미지를 진짜로 볼 확률이 0.57에서 0.43으로 떨어졌다는 뜻이고, 판별자가 조금씩 우세해지는 흐름입니다. GAN의 손실은 단조 감소하지 않으며, 손실이 균형처럼 보여도 품질은 낮을 수 있어서 PART 4의 지표로 따로 평가합니다.

지금 실행 중인 본인의 로그를 이 기준값과 비교해 보십시오. loss_d가 1.386 근처에 머무는지, 0 쪽으로 떨어지는지, loss_g가 0.693보다 커지는지를 보면 현재 어느 쪽이 우세한지 알 수 있습니다. 로그는 실행마다 다르므로 차트의 숫자와 같을 필요는 없고, 흐름의 방향을 보면 됩니다.

PART 3 · 계산 비용

계산 비용 : 학습에 35분이 걸리는 이유

회로 하나는 작지만($2^7 = 128$차원) 실행 횟수가 많음

회로 실행 수 = sub-generator 4 × batch 8 × step당 gen() 2회 × 400 step = 25,600회

회로 하나는 7큐빗(상태벡터 128차원)으로 작지만, 이 횟수만큼 forward와 backprop을 반복함.

미분 방식	어디서	샘플 하나의 생성자 기울기 비용	사용처
backprop	시뮬레이터만	forward 1번 + 역전파 (상태벡터 직접 미분)	2일차 QGAN (default.qubit)
adjoint	시뮬레이터만	forward 1번 + 역방향 1번, 메모리 적음	3일차 MediQ-GAN (lightning)
parameter-shift	하드웨어 가능	파라미터당 2번 → 280 × 2 = 560번 (× shot 수)	실제 양자 장치

19

[약 3분]
계산 비용입니다. 회로 실행 수는 sub-generator 4개, batch 8개, step당 gen() 2회, 400 step을 곱한 25,600회입니다. 회로 하나는 7큐빗, 상태벡터 128차원으로 작지만 이 횟수만큼 forward와 backprop을 반복하므로 35분이 걸립니다. 아래 표는 미분 방식 비교입니다. backprop은 상태벡터 전체를 미분하므로 시뮬레이터에서만 가능하고 2일차에서 씁니다. adjoint는 forward 한 번과 역방향 한 번으로 구하고 메모리가 적어서 3일차 MediQ-GAN의 lightning.qubit에서 씁니다. parameter-shift는 하드웨어에서도 가능한 방식으로, 회전각마다 θ ± π/2의 회로를 두 번 더 실행합니다. 생성자 파라미터가 280개이면 샘플 하나의 gradient에 560번이고, 각 실행에는 확률 추정을 위한 수천 shot이 필요합니다. README 실험에서 sub-generator가 적을수록 빠른 이유도 같습니다. 회로 호출 횟수, 즉 sub-generator 수 곱하기 batch가 비용을 지배합니다. parameter-shift의 비용을 숫자로 보면, 샘플 하나의 gradient에 회로 560번이 필요하고, 각 실행에서 확률을 1,024 shot으로 추정한다면 573,440번의 측정이 필요합니다. 여기에 batch와 step 수를 곱해야 학습 전체의 비용이 됩니다. 오늘 실습이 exact 시뮬레이션의 backprop으로만 35분 안에 끝나는 이유가 여기에 있습니다.

PART 3 · 한계

한계 : 실제와 생성 이미지의 픽셀 합 불일치

정규화 단위가 달라, 판별자가 형태가 아닌 밝기 총합만으로도 구분할 수 있음

	정규화 단위	이미지 한 장의 픽셀 합
실제 이미지 (real_flat)	이미지 전체	항상 1
생성 이미지 (gen(...))	patch 단위 (ancilla = 0 확률)	0 ~ 4 (Colab 실측 약 1.63)

```
real_flat = real_flat / real_flat.sum(dim=1, keepdim=True)   # sum = 1
fake      = gen(8)            # each of 4 patches sums to 0..1
```

수정 방법 (실습 후 과제)

실제 이미지도 patch별로 정규화

patch마다 합 1로 맞춰 생성 쪽과 단위를 맞춤

생성 patch를 최댓값으로 나눔

0 ~ 1 척도로 맞추고 실제 이미지는 그대로 둠 (PennyLane QGAN 튜토리얼 방식)

20

[약 2분]

한계 한 가지를 짚겠습니다. 실제 이미지는 코드에서 이미지 한 장의 픽셀 합을 1로 나눈 값이므로 장당 합이 항상 1입니다. 반면 생성 이미지는 patch마다 ancilla가 0일 확률이라 patch 합이 0에서 1 사이이고, 네 개를 이으면 0에서 4 사이가 됩니다. Colab 실행에서 생성 이미지의 장당 픽셀 합은 약 1.63이었습니다. 두 쪽의 단위가 다르다는 뜻입니다. 지표 계산에서는 to_unit_scale이 이미지별 min-max로 다시 맞추므로 지표 값에 직접 섞이지는 않고, 영향을 받는 것은 학습 과정입니다. 수정 방법은 두 가지입니다. 실제 이미지도 patch별로 정규화해서 생성 쪽과 단위를 맞추거나, 생성 patch를 최댓값으로 나눠 0과 1 척도로 맞추고 실제 이미지는 그대로 두는 PennyLane QGAN 튜토리얼 방식입니다. 이 저장소의 공개 수치는 현재 방식으로 학습한 결과이고, 고친 방식으로 다시 학습해 비교하는 것은 실습 후 과제로 적합합니다.

확인 질문입니다. 판별자가 형태가 아니라 밝기 총합만으로 진짜와 가짜를 구분할 수 있다면 학습에 어떤 문제가 생깁니까. 생성자는 병변의 모양을 배우는 대신 픽셀 합을 1에 맞추는 방향으로 먼저 움직일 수 있습니다. 그래서 이 한계는 지표 값보다 학습 과정에 영향을 줍니다.

PART 4 · 노트북 §6

생성과 평가

생성 이미지를 지표 4종으로 평가함. 3일차 MediQ-GAN에도 같은 지표를 씀

노트북 범위	§6 생성 이미지 확인과 지표 4종
다루는 코드	gm.to_unit_scale · mean_image_correlation · nearest_neighbor_distance · sample_diversity

21

[약 1분]

PART 4는 생성과 평가입니다. 노트북 §6에서 생성 이미지를 확인하고 지표 4종을 계산합니다. 다루는 코드는 gm.to_unit_scale, mean_image_correlation, nearest_neighbor_distance, sample_diversity입니다. 지표에 쓰이는 용어를 먼저 정의한 뒤 지표 4종을 읽겠습니다.

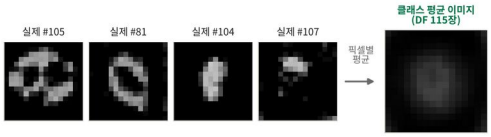
§6을 실행할 때 알아 둘 점입니다. 생성 이미지는 64장을 만들어 평가하고, 그림은 앞의 4장만 보여 줍니다. 평가 장수를 64장으로 둔 이유는 노트북 주석에 있습니다. 8장에서 16장 정도로 평가하면 클래스 평균 상관이 0.05에서 0.17까지 흔들려서 값을 비교할 수 없기 때문입니다. 이 모든 지표 함수는 practice/gen_metrics.py에 있으며, 3일차에도 같은 모듈을 그대로 씁니다.

PART 4 · 노트북 §6

지표 용어: 평균 이미지, 상관계수, L2 거리

지표 4종에 쓰이는 용어를 먼저 정의함. 모든 이미지는 16×16 = 256차원 벡터로 다룸

용어	정의	쓰이는 지표
픽셀 강도 histogram	픽셀 값(0 ~ 1)을 구간별로 나누어 개수를 센 분포	①
클래스 평균 이미지	실제 DF 이미지 115장을 픽셀별로 평균한 이미지. 클래스의 전형적인 형태	②
Pearson 상관계수	두 이미지 벡터에서 각각 평균을 뺀 뒤 코사인 유사도를 계산. -1 ~ 1이고, 1이면 밝기 패턴이 같은 모양	②
L2 거리	두 이미지 벡터 차이의 길이, $\sqrt{(\Sigma (\text{픽셀 차})^2)}$. 0이면 같은 이미지	③ ④
최근접 실제 거리	생성 이미지 하나에서 모든 실제 이미지까지 L2 거리의 최솟값. 0에 가까우면 학습 이미지를 외운 것(암기)	③
샘플 간 거리	생성 이미지 쌍의 L2 거리 평균. 0에 가까우면 모두 같은 이미지(mode collapse). 비교 기준은 실제 DF 쌍의 평균 2.505	④



왼쪽 네 장은 실제 DF 이미지 예이고, 오른쪽이 115장의 픽셀별 평균임. 지표 ②는 생성 이미지와 이 평균 이미지의 상관계수를 구함

22

[약 2분]

지표 4종에 쓰이는 용어를 먼저 정의합니다. 모든 이미지는 16×16, 즉 256차원 벡터로 다룹니다. 픽셀 강도 histogram은 픽셀 값을 구간별로 나누어 개수를 센 분포입니다. 클래스 평균 이미지는 실제 DF 이미지 115장을 픽셀별로 평균한 이미지로, 위 그림의 오른쪽이 그 결과이며 클래스의 전형적인 형태를 나타냅니다. Pearson 상관계수는 두 이미지 벡터에서 각각 평균을 뺀 뒤 코사인 유사도를 구한 값으로 −1에서 1 사이이고, 1이면 밝기 패턴이 같은 모양이라는 뜻입니다. L2 거리는 두 이미지 벡터 차이의 길이, 즉 픽셀 차이의 제곱합의 제곱근이며 0이면 같은 이미지입니다. 최근접 실제 거리는 생성 이미지 하나에서 모든 실제 이미지까지의 L2 거리 가운데 최솟값입니다. 0에 가까우면 학습 이미지를 그대로 외운 것, 즉 암기입니다. 샘플 간 거리는 생성 이미지 쌍의 L2 거리를 평균한 값이고, 0에 가까우면 모두 같은 이미지를 만드는 mode collapse입니다. 비교 기준은 실제 DF 쌍의 평균인 2.505입니다.

PART 4 · 노트북 § 6

생성품질 지표 4종

16×16 저해상도라 FID 대신 해석 가능한 지표를 씀 (practice/gen_metrics.py)

지표	보는 것	함수	읽는 법
① 픽셀 강도 분포	밝기 분포가 실제와 맞는가	plot_pixel_histogram	척도 변환 전 값도 함께 봄
② 클래스 평균 상관	병변의 전체 형태를 재현했는가	mean_image_correlation	높을수록 좋음. DF > NV여야 함
③ 최근접 실제 거리	학습 데이터를 외웠는가	nearest_neighbor_distance	0에 가까우면 암기
④ 샘플 간 거리	같은 그림만 반복하는가	sample_diversity	0에 가까우면 mode collapse

to_unit_scale(x, mode) - 세 척도를 0 ~ 1로 맞춘 뒤 지표를 계산함

mode	대상	변환
prob	QGAN 출력 (patch별 합 0 ~ 1인 확률)	이미지별 min-max
tanh	MediQ-GAN 출력 (-1 ~ 1)	(x + 1) / 2
unit	전처리 데이터 (0 ~ 1)	그대로

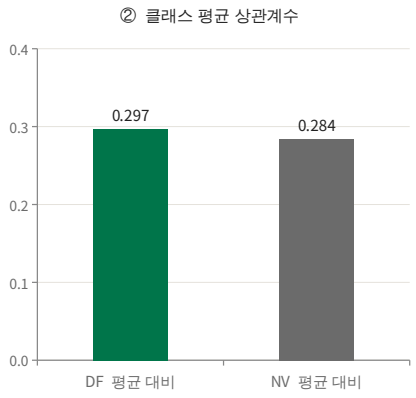
23

[약 2분]
지표 4종을 읽겠습니다. ① 픽셀 강도 분포는 밝기 분포가 실제와 맞는지 보는 것으로, 척도 변환 전의 값도 함께 봅니다. ② 클래스 평균 상관은 병변의 전체 형태를 재현했는지 보는 것으로, 높을수록 좋고 DF 평균 대비가 NV 평균 대비보다 높아야 합니다. ③ 최근접 실제 거리는 학습 데이터를 외웠는지 보는 것으로 0에 가까우면 암기입니다. ④ 샘플 간 거리는 같은 그림만 반복하는지 보는 것으로 0에 가까우면 mode collapse입니다. 16×16 저해상도에서는 FID를 쓰지 않습니다. FID는 Inception 특징을 쓰는데 이 해상도에서는 특징이 의미를 갖지 못하고, 표본도 수백 장 수준이기 때문입니다. 모든 지표는 to_unit_scale로 0과 1 사이로 맞춘 뒤 계산합니다. 아래 표의 prob는 QGAN 출력으로 이미지별 min-max, tanh는 MediQ-GAN 출력으로 x 더하기 1을 2로 나눈 값, unit은 전처리 데이터로 그대로 씁니다. ②는 평균 형태만 보므로 모든 샘플이 같은 평균 이미지여도 높게 나옵니다. ④와 함께 봐야 하고, ③이 0에 가까우면 학습 데이터를 복사한 것이라 증강 가치가 없습니다.
확인 질문입니다. ② 클래스 평균 상관이 높고 ④ 샘플 간 거리가 0에 가까우면 어떤 상태입니까.
생성자가 평균에 가까운 이미지 하나를 반복해서 만드는 mode collapse 상태입니다. 3일차의 MediQ-GAN 결과가 이 경향에 가깝습니다.

PART 4 · 결과 읽기

Colab 실행 결과

형태는 어느 정도 재현했지만 DF와 NV의 상관 차이는 작음. 실행마다 값이 조금씩 다름



지표	결과	해석
② 상관 (DF / NV 평균 대비)	0.297 / 0.284	DF와 NV 차이가 작음
③ 최근접 실제 거리	평균 1.579, 최솟값 0.956	0보다 충분히 커서 암기는 아님
④ 샘플 간 거리	1.731	실제 DF 2.505의 약 70% - mode collapse 아님

DF와 NV 상관이 거의 같음. 생성자가 DF 고유의 특징을 잡았는지, 16×16 그레이스케일에서 두 클래스의 평균 형태가 원래 비슷한지는 이 결과만으로 구분할 수 없음.

Colab에서 노트북을 끝까지 실행한 결과 (400 step, 생성 64장)

[약 2분]

Colab에서 노트북을 끝까지 실행한 결과입니다. 400 step으로 생성한 64장을 평가했습니다. ② 상관은 DF 평균 대비 0.297, NV 평균 대비 0.284로 차이가 작습니다. 생성자가 DF 고유의 특징을 잡았는지, 16×16 그레이스케일에서는 두 클래스의 평균 형태가 원래 비슷한지가 이 결과만으로는 구분되지 않습니다. 토론 포인트로 클래스 평균 이미지 두 장을 직접 비교해 보시기 바랍니다. ③ 최근접 실제 거리는 평균 1.579, 최솟값 0.956입니다. 최솟값이 0보다 충분히 크므로 암기는 아닙니다. ④ 샘플 간 거리는 1.731로 실제 DF의 2.505의 약 70%이므로 mode collapse는 아닙니다. 이 수치는 실행마다 조금씩 다르고, README의 600 step 결과, 즉 상관 0.195, 최근접 1.64, 다양성 1.92와도 차이가 있습니다.

본인 결과를 이 표와 비교할 때는 소수점보다 방향을 보십시오. ② 상관이 0보다 뚜렷하게 크고 DF 평균 대비가 NV 평균 대비보다 큰지, ③ 최솟값이 0보다 충분히 큰지, ④가 0으로 무너지지 않았는지를 확인하면 됩니다. 3일차의 QGAN 기준선은 강사 checkpoint 600 step으로 다시 계산하므로, 오늘 본인 값과는 다르게 나옵니다.

정리

2일차 정리

segmentation 전처리부터 QGAN 학습·평가까지

주제	정리
데이터와 전처리	DF 115장을 증강 대상으로 둠. K-means + FCM segmentation으로 병변만 남기고 16×16, 0 ~ 1로 축소 (INTER_AREA)
sub-generator 회로	7큐빗(데이터 6 + ancilla 1), RY encoding 후 [학습 RY + 인접 CZ] × 10. ancilla = 0인 확률 64개가 patch 하나
post-selection	ancilla = 0 항목만 쓰면 patch 밝기 총합이 0 ~ 1에서 자유로워져 배경 patch도 표현 가능
생성자·판별자	생성자 280개(양자 회전각) vs 판별자 19,201개. 학습률 0.05 / 0.001로 균형을 맞춤
학습과 손실	step마다 판별자 → 생성자 순서로 갱신. 기준값 loss_d 1.386, loss_g 0.693, loss_d → 0이면 판별자 완승
계산 비용	400 step에 회로 25,600회. backprop은 시뮬레이터 전용, 실제 장치는 parameter-shift(파라미터당 2회 × shot)
평가와 한계	지표 4종을 함께 봄. 실제(합 1)와 생성(patch별 합) 이미지의 픽셀 합 불일치, 16×16, shot noise 없음

25

[약 2분]

2일차 내용을 PART 순서대로 정리합니다. 데이터와 전처리에서는 DF 115장을 증강 대상으로 두고 K-means와 FCM segmentation으로 병변만 남겨 16×16, 0과 1 사이로 줄였습니다. 회로는 7큐빗, 즉 데이터 6개와 ancilla 1개이고 post-selection으로 patch 64개를 얻습니다. 생성자는 280개의 양자 회전각, 판별자는 19,201개 파라미터이며 학습률 0.05와 0.001로 균형을 맞췄습니다. 학습은 step마다 판별자, 생성자 순서로 갱신하고 기준값 1.386과 0.693으로 로그를 읽습니다. 비용은 400 step에 회로 25,600회이고 backprop은 시뮬레이터 전용입니다. 평가는 지표 4종을 함께 보며, 실제와 생성 이미지의 픽셀 합 불일치와 16×16 해상도, shot noise가 없다는 점을 한계로 기억해 주십시오. 3일차에는 같은 지표로 MediQ-GAN을 평가해 비교합니다.

마지막으로 7번 섹션의 저장 안내입니다. 학습한 checkpoint와 직접 전처리한 샘플을 Google Drive에 저장하며, 원본 6GB는 저장하지 않습니다. 권한 창이 뜨면 본인 계정을 고르고 모두 선택을 체크한 뒤 계속을 누릅니다. 일부만 허용하면 drive.mount가 실패하므로 셀을 다시 실행해 주십시오. 3일차는 저장소 자산으로 진행하므로 저장은 선택입니다. 3일차에는 노트북을 새로 열어 1번 섹션부터 실행하므로 오늘의 런타임을 유지할 필요는 없습니다.